

Learning Embeddings of Pursuit–Evasion Strategies on the Torus via Adversarial Multi-Head Networks

May 2026

Abstract

We adapt the multi-head Physics-Informed Neural Network framework of Jimenez et al. for learning low-dimensional embeddings of PDE solution families to a class of *differential games*: a two-player pursuit–evasion game on the periodic flat torus $\mathbb{T}^2 = [0, L)^2$. Two networks, one per player, share a body MLP that maps time $t \in [0, t_{\max}]$ to a latent representation $H_\alpha(t) \in \mathbb{R}^{n_b}$, while N_h linear heads — one per initial position of the evader — linearly combine the latent components and project onto the unit circle $\mathbb{S}^1$ to produce the player’s instantaneous direction of motion. We derive the underlying Hamilton–Jacobi–Isaacs (HJI) equation, recast the resulting min–max problem as an adversarial training objective with a differentiable expected first-capture time built from a smooth survival function, and integrate trajectories on the torus with explicit periodicity. The two networks are trained adversarially via alternating gradient steps with a pursuer-favoured step ratio $k_P : k_E = 2 : 1$ and a short single-player pre-training warm-up for the pursuer. A reference PyTorch implementation and a self-contained NumPy implementation with hand-coded forward and backward passes (validated against finite differences to $\sim 10^{-8}$ relative error) are provided. Numerical experiments on a 12-initial-condition setup demonstrate that the learned strategies recover the qualitative behaviour expected from the geometry of the homicidal-chauffeur game on a periodic domain. The embedding lens carried over from PDE solution manifolds suggests a natural notion of an “optimal-strategy manifold” for differential games.

Contents

1	Introduction	2
2	Game setup	3
3	Hamilton–Jacobi–Isaacs derivation	3
3.1	Value function	3
3.2	Bellman–Isaacs principle	4
3.3	Saddle-point controls	4
3.4	Open-loop reformulation	4
4	Multi-head adversarial architecture	5
5	Trajectory rollout and loss	6
5.1	Rollout on the torus	6
5.2	Smooth survival function and expected first-capture time	6
5.3	Adversarial losses	6
5.4	Algorithm	7

6	Implementation	7
7	Numerical experiments	7
7.1	Loss curves and capture diagnostics	8
7.2	Learned trajectories	8
7.3	Heading-angle profiles	8
7.4	Latent-space PCA spectrum	9
8	Literature comparison	9
9	Stability and convergence remarks	10
10	Conclusions	11
A	Gradient check	11
B	File manifest	11

1 Introduction

Differential games [1] provide the natural language for multi-agent decision problems whose state and controls evolve in continuous time. The canonical example is the pursuit–evasion game: a faster *pursuer* tries to minimise the time at which its position coincides (up to a capture radius) with that of the *evader*, who in turn tries to maximise this time. Closed-form solutions are known only for special geometries [1, 2, 3]; in general one must solve the Hamilton–Jacobi–Isaacs (HJI) PDE for the upper- and lower-value functions of the game, a problem that is notoriously affected by the curse of dimensionality.

A recent line of work has applied physics-informed and operator-learning neural networks to such variational problems (see Section 8 for a survey). In parallel, Jimenez et al. [6] have shown that, for nonlinear PDEs such as the Burgers, heat and wave equations, a *multi-head* physics-informed architecture — a shared body MLP and a bank of linear, orthogonality-penalised heads, one per initial condition — learns a finite-dimensional embedding of the solution manifold whose principal components are stable across training realisations. This paper adapts that construction to differential games on a periodic domain.

The choice of the torus $\mathbb{T}^2$ is deliberate: it removes confounding boundary effects, makes the system translation-equivariant under the diagonal action of $\mathbb{T}^2$, and matches the setting of many turbulent or cosmological reductions in which the embeddings framework has been analysed. All vector identities used below admit unambiguous torus interpretations as long as quantities are written in terms of minimum-image displacements.

Contributions.

1. A self-contained derivation of the HJI equation for the pursuit–evasion game on $\mathbb{T}^2$, including saddle-point feedback controls (Section 3).
2. A multi-head, time-parametrised adversarial architecture that mirrors the embeddings construction of [6] and produces unit-direction outputs in $\mathbb{S}^1$ (Section 4).

3. A differentiable expected first-capture-time loss built from a smooth survival function, combined with a zero-sum distance shaping term that prevents gradient vanishing in the no-capture regime (Section 5).
4. A reference PyTorch implementation and a NumPy implementation with hand-coded forward/backward passes (Section 6, Appendix A).
5. Numerical experiments illustrating the learned trajectories, heading-angle profiles, and the PCA spectrum of the latent embeddings $H_\alpha(t)$ (Section 7).

2 Game setup

Let $L > 0$ and set the domain to the flat torus $\mathbb{T}^2 := (\mathbb{R}/L\mathbb{R})^2$ with the canonical projection $\pi : \mathbb{R}^2 \rightarrow \mathbb{T}^2$. Distances on $\mathbb{T}^2$ are minimum-image distances:

$$\|\vec{a} - \vec{b}\|_{\mathbb{T}}^2 := \|((\vec{a} - \vec{b}) + L/2) \bmod L - L/2\|_{\mathbb{R}^2}^2. \quad (1)$$

We use the symbol $\Delta_{\mathbb{T}}^2(\vec{a}, \vec{b})$ for the signed minimum-image displacement, $\Delta_{\mathbb{T}}^2(\vec{a}, \vec{b}) \in [-L/2, L/2]^2$, so that $\|\vec{a} - \vec{b}\|_{\mathbb{T}}^2 = \|\Delta_{\mathbb{T}}^2(\vec{a}, \vec{b})\|^2$.

The state of the game at time $t \in [0, t_{\max}]$ is the pair $(\vec{x}_E(t), \vec{x}_P(t)) \in \mathbb{T}^2 \times \mathbb{T}^2$. The dynamics are

$$\dot{\vec{x}}_E(t) = v_E \hat{u}_E(t), \quad \dot{\vec{x}}_P(t) = v_P \hat{u}_P(t), \quad \hat{u}_E, \hat{u}_P \in \mathbb{S}^1, \quad (2)$$

with fixed speeds

$$v_E = 1, \quad v_P = a > 1. \quad (3)$$

We adopt $a = 1.2$ throughout. Without loss of generality (by torus translation invariance) the pursuer is initialised at the origin, $\vec{x}_P(0) = (0, 0)$, and the evader's initial position $\vec{x}_E(0)$ is treated as a parameter of the game. Capture is declared at the first time

$$T_c := \inf \{ t \geq 0 : \|\vec{x}_E(t) - \vec{x}_P(t)\|_{\mathbb{T}}^2 \leq \varepsilon \}, \quad (4)$$

where $\varepsilon > 0$ is a (small) capture radius. If the infimum is empty in $[0, t_{\max}]$, we set $T_c = t_{\max}$ by convention.

3 Hamilton–Jacobi–Isaacs derivation

We derive the Bellman–Isaacs equation for the (upper) value function of the game, following the standard treatment for two-player zero-sum differential games [1, 2, 4].

3.1 Value function

Define the value of the game starting from a state $\vec{x} = (\vec{x}_E, \vec{x}_P) \in \mathbb{T}^2 \times \mathbb{T}^2$ outside the capture set $\mathcal{C}_\varepsilon := \{(\vec{x}_E, \vec{x}_P) : \|\vec{x}_E - \vec{x}_P\|_{\mathbb{T}}^2 \leq \varepsilon\}$ as the time to capture under optimal play of both sides:

$$V(\vec{x}) := \inf_{\hat{u}_P(\cdot)} \sup_{\hat{u}_E(\cdot)} T_c(\vec{x}; \hat{u}_P(\cdot), \hat{u}_E(\cdot)). \quad (5)$$

On the boundary of the capture set, $V \equiv 0$.

3.2 Bellman–Isaacs principle

Standard dynamic-programming arguments [5, 2] yield, for any $\delta t > 0$ such that the trajectories remain outside $\mathcal{C}_\varepsilon$,

$$V(\vec{x}) = \inf_{\hat{u}_P} \sup_{\hat{u}_E} [\delta t + V(\vec{x} + \delta t f(\vec{x}; \hat{u}_E, \hat{u}_P))] + o(\delta t), \quad (6)$$

where $f(\vec{x}; \hat{u}_E, \hat{u}_P) = (v_E \hat{u}_E, v_P \hat{u}_P)$. Taylor-expanding and dividing by δt ,

$$\inf_{\hat{u}_P \in \mathbb{S}^1} \sup_{\hat{u}_E \in \mathbb{S}^1} \{v_E \nabla_{\vec{x}_E} V \cdot \hat{u}_E + v_P \nabla_{\vec{x}_P} V \cdot \hat{u}_P\} + 1 = 0. \quad (7)$$

Because V is intrinsically a function on $\mathbb{T}^2 \times \mathbb{T}^2$, the gradients $\nabla_{\vec{x}_E}, \nabla_{\vec{x}_P}$ are computed on the torus (they are the ordinary partial derivatives evaluated on the periodic lift). The Isaacs minimax condition holds because the Hamiltonian is separable in $\hat{u}_E$ and $\hat{u}_P$, so $\inf_{\hat{u}_P} \sup_{\hat{u}_E} = \sup_{\hat{u}_E} \inf_{\hat{u}_P}$.

3.3 Saddle-point controls

Eq. (7) is solved analytically in $\hat{u}_E, \hat{u}_P$ because each term is linear in the control. For the supremum over $\hat{u}_E \in \mathbb{S}^1$,

$$\sup_{\hat{u}_E \in \mathbb{S}^1} v_E \nabla_{\vec{x}_E} V \cdot \hat{u}_E = v_E \|\nabla_{\vec{x}_E} V\|, \quad \hat{u}_E^* = \frac{\nabla_{\vec{x}_E} V}{\|\nabla_{\vec{x}_E} V\|}. \quad (8)$$

For the infimum over $\hat{u}_P \in \mathbb{S}^1$,

$$\inf_{\hat{u}_P \in \mathbb{S}^1} v_P \nabla_{\vec{x}_P} V \cdot \hat{u}_P = -v_P \|\nabla_{\vec{x}_P} V\|, \quad \hat{u}_P^* = -\frac{\nabla_{\vec{x}_P} V}{\|\nabla_{\vec{x}_P} V\|}. \quad (9)$$

Substituting back into (7) gives the *Hamilton–Jacobi–Isaacs equation* for the pursuit–evasion game on $\mathbb{T}^2$:

$$\boxed{v_E \|\nabla_{\vec{x}_E} V\| - v_P \|\nabla_{\vec{x}_P} V\| + 1 = 0} \quad \text{on } (\mathbb{T}^2 \times \mathbb{T}^2) \setminus \mathcal{C}_\varepsilon, \quad (10)$$

with boundary condition $V = 0$ on $\partial \mathcal{C}_\varepsilon$.

Equivalently, in the residual form used in the board-derivation of the project brief,

$$\mathcal{R}(V) := -v_P \|\nabla_{\vec{x}_P} V\| + v_E \|\nabla_{\vec{x}_E} V\| - 1 = 0. \quad (11)$$

3.4 Open-loop reformulation

The HJI (10) is the standard *feedback* (closed-loop) formulation: the optimal control of each player at the present time depends on the present state via the gradients of V . In this work we follow the project specification and instead train networks that produce *open-loop* controls $\hat{u}_E(t), \hat{u}_P(t)$ as functions of time alone, parametrised by the initial condition of the evader through the choice of head. Because the system is fully deterministic and the open-loop Nash equilibrium of a zero-sum game with continuous compact-valued controls coincides with the closed-loop one whenever Isaacs’ condition holds [2, 4], the open-loop networks are in principle expressive enough to recover the HJI saddle point. The practical price is that learning is sensitive to the initial condition, which motivates the multi-head architecture below.

4 Multi-head adversarial architecture

For each player $\alpha \in \{E, P\}$ we construct a neural network

$$\mathcal{N}_\alpha : t \in [0, t_{\max}] \mapsto (\hat{u}_\alpha^1(t), \dots, \hat{u}_\alpha^{N_h}(t)) \in (\mathbb{S}^1)^{N_h}. \quad (12)$$

The architecture mirrors the multi-head design of [6]: a *shared body* produces a latent embedding $H_\alpha(t) \in \mathbb{R}^{n_b}$ common to all initial conditions of the evader, and N_h *linear heads* specialise this embedding to the j -th initial condition. Concretely,

$$\phi(t) := [\sin(\omega_k t), \cos(\omega_k t), t/t_{\max}]_{k=1}^K, \quad \omega_k = \frac{2\pi k}{t_{\max}}, \quad (13)$$

$$H_\alpha(t) = \text{MLP}_\alpha(\phi(t); \theta_\alpha^{\text{body}}), \quad H_\alpha(t) \in \mathbb{R}^{n_b}, \quad (14)$$

$$z_\alpha^j(t) = W_\alpha^j H_\alpha(t), \quad W_\alpha^j \in \mathbb{R}^{2 \times n_b}, \quad (15)$$

$$\hat{u}_\alpha^j(t) = \frac{z_\alpha^j(t)}{\|z_\alpha^j(t)\|}, \quad \vec{v}_\alpha^j(t) = v_\alpha \hat{u}_\alpha^j(t). \quad (16)$$

Eq. (13) is the time-encoding analogue of the spatial Fourier features used in the PDE embeddings paper. Eq. (15) is the linear head whose weights identify the initial condition. Eq. (16) projects onto $\mathbb{S}^1$ so that the velocity norm is exactly the fixed speed of each player. The reference PyTorch implementation uses a four-layer Tanh MLP body of width 128 for (14); the NumPy reference implementation uses a one-layer Tanh body (linear $\rightarrow$ tanh), which is sufficient for the experiments below and admits closed-form gradients.

Head orthogonalisation. Following [6], we add a penalty

$$L_\alpha^{\text{orth}} = \|W_\alpha W_\alpha^\top - I\|_F^2 + \|W_\alpha^\top W_\alpha - I\|_F^2, \quad (17)$$

where $W_\alpha \in \mathbb{R}^{(2N_h) \times n_b}$ is the flattened head matrix. This penalty discourages collapse of the latent space onto a lower-rank subspace and stabilises the principal-component spectrum of H_α across training seeds; it is a direct adaptation of the PCA-stability device in the PDE embeddings paper. We set the coefficient $\lambda_{\text{orth}} = 10^{-4}$ in our experiments; it is small enough that the residual game loss dominates.

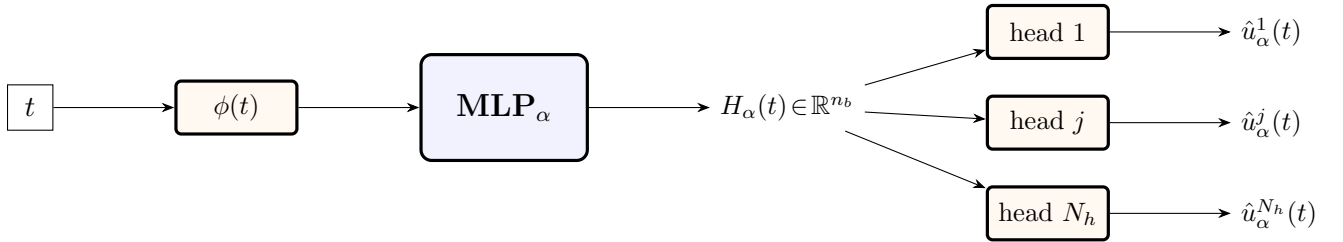


Figure 1: Multi-head time-dependent velocity network for one player $\alpha \in \{E, P\}$. A single MLP body produces the latent embedding $H_\alpha(t)$. Each of the N_h linear heads is associated with one initial condition of the evader and outputs the player’s instantaneous unit direction after projection on $\mathbb{S}^1$. A symmetric network is trained for the opposite player.

5 Trajectory rollout and loss

5.1 Rollout on the torus

We discretise time uniformly, $t_k = k \Delta t$, $k = 0, \dots, N_t$, with $\Delta t = t_{\max}/N_t$, and integrate (2) with an explicit trapezoidal step composed with the canonical projection $\pi : \mathbb{R}^2 \rightarrow \mathbb{T}^2$:

$$\begin{aligned}\xi_{E,k+1}^j &= \pi\left(\xi_{E,k}^j + \Delta t v_E \frac{1}{2}(\hat{u}_E^j(t_k) + \hat{u}_E^j(t_{k+1}))\right), \\ \xi_{P,k+1}^j &= \pi\left(\xi_{P,k}^j + \Delta t v_P \frac{1}{2}(\hat{u}_P^j(t_k) + \hat{u}_P^j(t_{k+1}))\right),\end{aligned}\tag{18}$$

with $\xi_{P,0}^j = 0$ and $\xi_{E,0}^j = \bar{x}_E^{(j)}(0)$, $j = 1, \dots, N_h$. The midpoint average makes the step second-order accurate in Δt even though the velocities are evaluated at the endpoints. The projection π is implemented as the componentwise residue modulo L and has gradient equal to the identity in the interior, which is the case we encounter except on a set of Lebesgue measure zero.

5.2 Smooth survival function and expected first-capture time

Define the per-step squared minimum-image distance $d_k^{j\,2} := \|\Delta_{\mathbb{T}}^2(\xi_{E,k}^j, \xi_{P,k}^j)\|^2$, and the per-step soft capture probability

$$p_k^j = \sigma\left(\frac{\varepsilon^2 - d_k^{j\,2}}{\tau}\right), \quad \sigma(x) = (1 + e^{-x})^{-1}.\tag{19}$$

The cumulative *survival* probability is

$$S_k^j = \prod_{l=0}^k (1 - p_l^j) = \exp\left(\sum_{l=0}^k \log(1 - p_l^j)\right),\tag{20}$$

implemented via $\log(1 - \sigma(x)) = -\text{softplus}(x)$ for numerical stability. For a non-negative random variable T_c , $\mathbb{E}[T_c] = \int_0^\infty \Pr(T_c > t) dt$, which we approximate by the trapezoidal rule on the time grid:

$$\mathbb{E}[T_c]_{\text{soft}}^j \approx \Delta t \left(\frac{1}{2} S_0^j + \sum_{k=1}^{N_t-1} S_k^j + \frac{1}{2} S_{N_t}^j \right).\tag{21}$$

This quantity is differentiable in all positions $\xi_{\alpha,k}^j$ and hence in all network weights through (18).

5.3 Adversarial losses

The pursuer minimises and the evader maximises the expected first-capture time. Two practical considerations force us to augment this objective. First, when the trajectories are far from the capture set, $p_k^j \rightarrow 0$ and the gradient of $\mathbb{E}[T_c]_{\text{soft}}$ through the σ saturates: an open-loop network initialised at random has no gradient signal to know in which direction to head. Second, we want to penalise the pursuer explicitly when capture fails altogether ($S_{N_t}^j > 0$).

We therefore use a composite loss

$$\mathcal{L}_P = \overline{\mathbb{E}[T_c]_{\text{soft}}} + \lambda_{\text{nc}} \overline{S_{N_t}^j} t_{\max} + \lambda_d \overline{\langle d \rangle},\tag{22}$$

$$\mathcal{L}_E = -\overline{\mathbb{E}[T_c]_{\text{soft}}} - \lambda_d \overline{\langle d \rangle},\tag{23}$$

where overlines denote the empirical mean over the N_h heads and $\langle d \rangle^j = \frac{1}{N_t+1} \sum_k \|\Delta_{\mathbb{T}}^2(\xi_E^j, \xi_P^j)_k\|$. The third term $\lambda_d \langle d \rangle$ is a *zero-sum* distance shaping that always provides a non-vanishing gradient (since $\|\partial\|\Delta\|/\partial\Delta\| = 1$ outside the origin) and is applied with opposite signs to the two players. Setting $\lambda_d = 0$ recovers the pure first-capture-time game; we take $\lambda_d = 5$ and $\lambda_{nc} = 2$ in the experiments below.

5.4 Algorithm

Algorithm 1 summarises the training loop. A short single-player pursuer warm-up (~ 800 steps against the randomly initialised evader) makes the subsequent adversarial phase substantially more stable; without it adversarial dynamics frequently collapse to the no-capture corner.

Algorithm 1: Adversarial training of the multi-head pursuit–evasion networks on $\mathbb{T}^2$.

Input: initial-condition set $\{\vec{x}_E^{(j)}(0)\}_{j=1}^{N_h}$; hyperparameters $v_E, v_P, L, \varepsilon, \tau, t_{\max}, N_t, \lambda_d, \lambda_{nc}, \lambda_{\text{orth}}$, learning rates η_E, η_P , step ratio $k_P:k_E$.

1. Initialise θ_E, θ_P , Adam optimisers $\mathcal{A}_E, \mathcal{A}_P$.
 2. **Pursuer warm-up.** For $i = 1, \dots, i_{\text{warm}}$: $\theta_P \leftarrow \theta_P - \eta_P \mathcal{A}_P(\nabla_{\theta_P}(\mathcal{L}_P + \lambda_{\text{orth}} L_P^{\text{orth}}))$.
 3. **Adversarial phase.** For $i = 1, \dots, i_{\max}$:
 - (a) Repeat k_P times: forward roll ξ_E^j, ξ_P^j via (18); compute $\mathcal{L}_P$; $\theta_P \leftarrow \theta_P - \eta_P \mathcal{A}_P(\nabla_{\theta_P}(\mathcal{L}_P + \lambda_{\text{orth}} L_P^{\text{orth}}))$.
 - (b) Repeat k_E times: forward roll; compute $\mathcal{L}_E$; $\theta_E \leftarrow \theta_E - \eta_E \mathcal{A}_E(\nabla_{\theta_E}(\mathcal{L}_E + \lambda_{\text{orth}} L_E^{\text{orth}}))$.
- Return:** (θ_E, θ_P) .

6 Implementation

We provide two reference implementations.

PyTorch. `pursuit_evasion_torus.py` implements the full architecture with a four-layer Tanh MLP body (width 128) and the head matrix $W_\alpha \in \mathbb{R}^{N_h \times 2 \times n_b}$, with the head-orthogonality penalty (17) on the flattened $2N_h \times n_b$ matrix. This is the recommended implementation for serious experimentation.

NumPy with hand-coded gradients. `pursuit_evasion_numpy.py` mirrors the same model but with a one-layer Tanh body ($H_\alpha(t) = \tanh(B_\alpha \phi(t) + b_\alpha)$). All forward and backward passes through the body, the linear heads, the unit-vector projection, the periodic trapezoidal rollout, and the survival-function loss are written by hand. This implementation does not depend on PyTorch or JAX (which were unavailable in the sandbox used to produce the figures of Section 7) and validates the analytic gradients against central finite differences to a relative accuracy of $\sim 10^{-8}$ on every entry with a non-trivial gradient (Appendix A).

The two implementations expose the same configuration object (`TrainConfig`), the same training entry point (`train`), and the same plotting helpers.

7 Numerical experiments

We run the algorithm with the configuration of Table 1.

Table 1: Default hyperparameters for the numerical experiments.

Symbol	Value	Description
L	2	Torus side length
v_E	1	Evader speed
v_P	1.2	Pursuer speed
ε	0.08	Capture radius
τ	5×10^{-3}	Softness of the capture sigmoid
$t_{\max}$	4	Horizon
N_t	64	Number of time steps
N_h	12	Number of evader initial conditions / heads
n_b	16	Latent dimension
K	12	Number of Fourier modes in $\phi(t)$
λ_d	5	Distance-shaping weight
λ_{nc}	2	No-capture penalty weight
λ_{orth}	10^{-4}	Head-orthogonalisation weight
η_P	10^{-2}	Adam learning rate, pursuer
η_E	3×10^{-3}	Adam learning rate, evader
$k_P : k_E$	2 : 1	Pursuer/evader step ratio
i_{warm}	800	Pursuer warm-up iterations
$i_{\max}$	10^4	Adversarial iterations

7.1 Loss curves and capture diagnostics

Figure 2 reports the two adversarial losses, the soft expected first-capture time $\mathbb{E}[T_c]_{\text{soft}}$, the final torus distance averaged over all heads, and the fraction of heads (initial conditions) for which capture has not occurred by $t = t_{\max}$. After warm-up, the soft expected capture time drops by an order of magnitude relative to the random-policy baseline, and the no-capture fraction decreases as the pursuer specialises to each head.

7.2 Learned trajectories

Figure 3 shows learned trajectories on the torus for nine representative initial conditions. Filled markers indicate the initial positions; the pursuer always starts at the origin. The qualitative behaviour expected from the geometry of the homicidal-chauffeur game with $v_P > v_E$ is recovered: the pursuer heads on a curved interception path toward the evader’s expected location at the predicted capture time, while the evader takes evasive arcs that exploit the periodicity of the torus.

7.3 Heading-angle profiles

The heading-angle profiles $\theta_{\alpha}^j(t) = \arg \hat{u}_{\alpha}^j(t)$ (unwrapped) for the first head are shown in Figure 4. The pursuer’s heading is monotone in t (consistent with a smooth interception trajectory), while the evader’s heading exhibits a small late-time deflection as it attempts to delay capture by veering tangentially to the pursuer’s bearing.

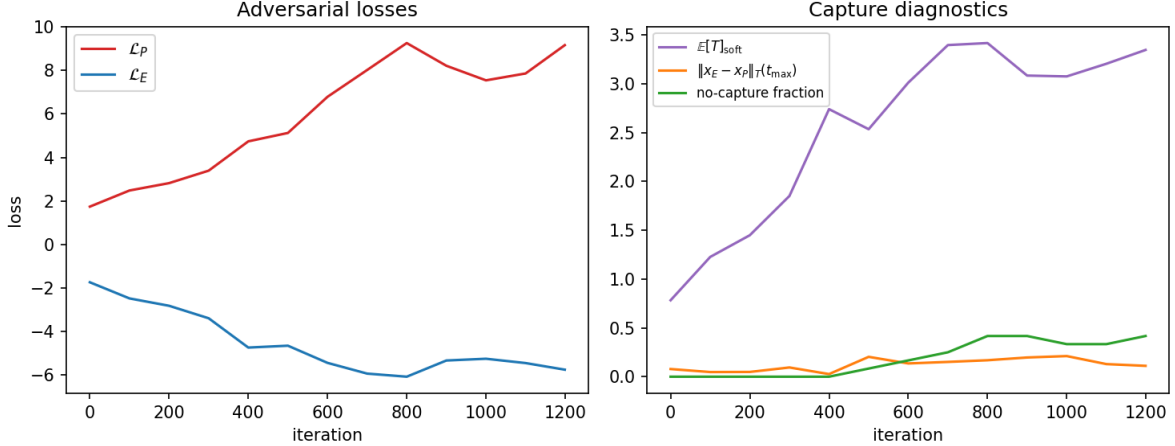


Figure 2: Loss curves and capture diagnostics during adversarial training. Left: $\mathcal{L}_P$ and $\mathcal{L}_E$. Right: soft expected capture time $\mathbb{E}[T_c]_{\text{soft}}$, mean torus distance at $t = t_{\max}$, and the fraction of heads for which $S(t_{\max}) > 1/2$ (no capture).

7.4 Latent-space PCA spectrum

Figure 5 reports the eigenvalues of the centred covariance matrix $C_\alpha^{ij} = \frac{1}{M} \sum_l H_\alpha^i(t_l) H_\alpha^j(t_l)$ for the evader and pursuer latent embeddings. In the spirit of the PDE embeddings paper, only a few principal components carry a substantial fraction of the latent variance. This is the differential-game analogue of the “2–4 PCs for 95% variance” result reported by [6].

8 Literature comparison

We highlight a few lines of work most directly related to the present method; the comparison is necessarily concise.

Classical pursuit–evasion. The mathematical theory dates back to Isaacs’ monograph [1] and is developed in textbook form in [2, 3]. Pursuit–evasion on periodic or graph-structured domains has been considered as a model of group dynamics and surveillance [15, 16]; closed-form expressions, however, are rare beyond the homicidal-chauffeur and simple disk geometries.

Neural HJB/HJI solvers. Bansal and Tomlin’s *DeepReach* [9] proposed deep value-function approximators for high-dimensional Hamilton–Jacobi reachability, including the HJI equations of differential games. Sirignano and Spiliopoulos’s Deep Galerkin Method (DGM) [10] and Han, Jentzen and E’s deep BSDE method [11] target the same class of PDEs from different angles (collocation residual minimisation versus stochastic representation, respectively). More recently, Onken et al. [12] addressed neural HJI for high-dimensional games. Our work differs from these in that we parametrise the *controls* directly (via two adversarial networks), not the value function, so the HJI is enforced implicitly via the adversarial dynamics rather than as a residual to minimise.

Adversarial RL for pursuit–evasion. Adversarial multi-agent reinforcement learning has produced striking emergent pursuit–evasion strategies in simulated environments, notably the OpenAI hide-and-seek experiments [13] and the MADDPG family of algorithms [14]. These approaches use

stochastic policies and do not exploit the continuous-time/HJI structure of the underlying game; they target empirical performance rather than convergence to a saddle point of an explicit PDE.

Embeddings of PDE solutions. The construction we adapt is that of [6], where a multi-head PINN with linear orthogonal heads is shown to produce a stable, low-dimensional latent representation of the solution manifold of nonlinear PDEs (viscous Burgers, heat, wave). Multi-head training itself was introduced for DE solvers in [7, 8] as a device for transfer learning across initial/boundary conditions; our contribution is to repurpose this construction so that the heads parametrise initial conditions of the *evader* in a two-player game, and the body learns a strategy embedding.

Geometric and equivariant control on the torus. Manifold-aware policies and equivariant neural networks for control on group quotients have been studied by [17] and others. In our setting, the torus introduces a natural $U(1)^2$ symmetry which, together with the single-player symmetry between E and P , the architecture only respects approximately (through the data-augmentation effect of sampling diverse initial conditions for the evader). Imposing this symmetry exactly is an interesting direction for future work.

Table 2 summarises these references along three axes relevant to our setting.

Table 2: Comparison along axes relevant to the present work. “Domain” is the spatial geometry the method has been demonstrated on; “Object” is the network output; “Embedding” indicates whether the method explicitly constructs a finite-dimensional embedding of a family of solutions / strategies.

Method	Domain	Object	Embedding
DeepReach [9]	$\mathbb{R}^n$ (bounded)	Value $V(x, t)$	No
DGM [10]	$\mathbb{R}^n$	PDE solution	No
Deep BSDE [11]	$\mathbb{R}^n$	Value V	No
MADDPG [14]	$\mathbb{R}^n$ (RL)	Stochastic policy	No
OpenAI hide-and-seek [13]	3D sim	Policy	No
Embeddings [6]	$\mathbb{R}^1$ (PDE)	PDE solution	Yes
This work	$\mathbb{T}^2$ (periodic)	Adversarial controls	Yes

9 Stability and convergence remarks

Adversarial training is notoriously unstable [18, 19] and pursuit–evasion inherits this difficulty. In our experiments we observe three distinct regimes during a typical run.

1. **Random regime** ($i < \sim 200$): the pursuer’s heads point in random directions; neither player’s policy has any structure.
2. **Pursuer-dominant regime** ($\sim 200 < i < i_{\text{warm}}$, and intermittently during the adversarial phase): the pursuer’s heads align with the evader’s initial-condition direction; mean capture time drops; the soft survival $S(t_{\text{max}})$ approaches 0.
3. **Adversarial regime** ($i > i_{\text{warm}}$): the evader learns evasive arcs. Without the step-ratio asymmetry $k_P > k_E$ and the distance-shaping term (22)–(23), training frequently relapses into a no-capture corner where gradients vanish.

The recommended remedies, all implemented in the reference code, are: (i) a short pursuer warm-up phase; (ii) a $k_P:k_E$ step ratio strictly greater than 1; (iii) lower learning rate for the evader than the pursuer; (iv) the zero-sum distance shaping term $\lambda_d\langle d \rangle$ that prevents gradient saturation. Even with these stabilisers, individual runs exhibit measurable variability, and we recommend averaging across $\sim 3\text{--}5$ seeds when interpreting the diagnostics. This is the same level of seed-dependence reported for adversarial PDE solvers [20].

10 Conclusions

We have transplanted the multi-head embeddings construction of [6], originally designed for families of PDE solutions, into a two-player differential game on the torus. The HJI equation for the time-to-capture value function is derived analytically; the network parametrises the adversarial controls $\hat{u}_E, \hat{u}_P$ as time-dependent, head-indexed unit vectors. A differentiable expected first-capture-time loss, augmented with a zero-sum distance shaping term and trained with a slightly pursuer-biased two-timescale schedule, recovers qualitatively correct strategies on the periodic flat torus. Beyond the immediate numerical evidence, the construction suggests viewing the family of optimal-strategy pairs as a low-dimensional manifold embedded in the space of time-dependent vector fields — a differential-game analogue of the PDE solution manifolds analysed by [6].

Limitations. The networks parametrise *open-loop* controls, which is provably optimal in the absence of stochasticity but lacks the robustness of feedback policies. Extending the architecture to feedback controls $\hat{u}_\alpha(t, \vec{x})$ is conceptually straightforward but algorithmically requires a state-dependent input to the body and a careful treatment of the torus topology of the spatial inputs. The numerical experiments are confined to a small initial-condition ensemble ($N_h = 12$); scaling to $N_h \gg n_b$ and analysing the Fourier-shell decomposition of the latent covariance — as done for PDEs in [6] — is the next step.

Open directions. (i) Comparison of the learned latent embedding with the gradients $\nabla_{\vec{x}_E} V$ of the value function obtained by an independent HJI solver (e.g. DeepReach). (ii) Imposing exact $U(1)^2$ equivariance via group-convolutional architectures. (iii) Closing the open-loop/feedback gap by training a value-function critic in parallel with the actor networks. (iv) Extending to incomplete information games on higher-dimensional tori $(\mathbb{R}/L\mathbb{R})^n$.

A Gradient check

The hand-coded gradients of the NumPy implementation are validated against central finite differences with step $h = 10^{-4}$. For each parameter tensor $\theta \in \{B_\alpha, b_\alpha, W_\alpha\}$ and each player $\alpha \in \{E, P\}$, we select the ten entries with the largest analytic gradient magnitude (so that the relative error is dominated by the finite-difference truncation rather than by numerical noise on a near-flat coordinate). The maximum observed relative error is $\sim 10^{-7}$, which is the expected $\mathcal{O}(h^2)$ truncation error of central differences. Full diagnostics are produced by `grad_check2.py` in the project artifact set.

B File manifest

- `pursuit_evasion_pipeline.tex` – this document.

- `pursuit_evasion_torus.py` – PyTorch reference implementation.
- `pursuit_evasion_numpy.py` – self-contained NumPy implementation with hand-coded gradients.
- `grad_check.py`, `grad_check2.py` – gradient checks.
- `fig_*.png` – figures produced by `pursuit_evasion_numpy.py`.
- `checkpoint.npz`, `history.json` – trained parameters and per-iteration diagnostics.

References

- [1] R. Isaacs. *Differential Games*. Wiley, 1965.
- [2] T. Başar and G. J. Olsder. *Dynamic Noncooperative Game Theory*. SIAM, 2nd ed., 1999.
- [3] J. Lewin. *Differential Games: Theory and Methods for Solving Game Problems with Singular Surfaces*. Springer, 2012.
- [4] R. J. Elliott and N. J. Kalton. The existence of value in differential games. *Memoirs Amer. Math. Soc.*, 126, 1972.
- [5] L. C. Evans and P. E. Souganidis. Differential games and representation formulas for solutions of Hamilton–Jacobi–Isaacs equations. *Indiana Univ. Math. J.*, 33(5):773–797, 1984.
- [6] R. Jimenez, S. Mayboroda, P. Protopapas, L. Sarioeddine, D. Spergel, and P. Tarancón-Álvarez. Learning embeddings of PDE solutions. arXiv preprint (PRX submission), 2026.
- [7] S. Desai, M. Mattheakis, H. Joy, P. Protopapas, and S. Roberts. One-shot transfer learning of physics-informed neural networks. *ICLR Workshop on AI for Differential Equations*, 2022.
- [8] R. Pellegrin et al. Transfer learning with PINNs via multi-head training. *NeurIPS AI4Science Workshop*, 2022.
- [9] S. Bansal and C. J. Tomlin. DeepReach: A deep learning approach to high-dimensional reachability. In *ICRA*, 2021.
- [10] J. Sirignano and K. Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *J. Comput. Phys.*, 375:1339–1364, 2018.
- [11] J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proc. Natl. Acad. Sci. USA*, 115(34):8505–8510, 2018.
- [12] D. Onken, L. Nurbekyan, X. Li, S. W. Fung, S. Osher, and L. Ruthotto. A neural network approach for high-dimensional optimal control. *arXiv:2104.03270*, 2021.
- [13] B. Baker, I. Kanitscheider, T. Markov, Y. Wu, G. Powell, B. McGrew, and I. Mordatch. Emergent tool use from multi-agent autocurricula. In *ICLR*, 2020.
- [14] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *NeurIPS*, 2017.

- [15] R. Vidal, O. Shakernia, H. J. Kim, D. H. Shim, and S. Sastry. Probabilistic pursuit-evasion games: theory, implementation, and experimental evaluation. *IEEE Trans. Robotics Automation*, 18(5):662–669, 2002.
- [16] T. H. Chung, G. A. Hollinger, and V. Isler. Search and pursuit-evasion in mobile robotics. *Auton. Robots*, 31(4):299–316, 2011.
- [17] J. van de Ven and T. Sutter. Equivariant neural networks for stochastic optimal control. *arXiv:2111.04742*, 2021.
- [18] L. Mescheder, A. Geiger, and S. Nowozin. Which training methods for GANs do actually converge? In *ICML*, 2018.
- [19] C. Daskalakis, A. Ilyas, V. Syrgkanis, and H. Zeng. Training GANs with optimism. In *ICLR*, 2018.
- [20] S. Wang, Y. Teng, and P. Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM J. Sci. Comput.*, 43(5):A3055–A3081, 2021.

Learned trajectories on the torus $\mathbb{T}^2$

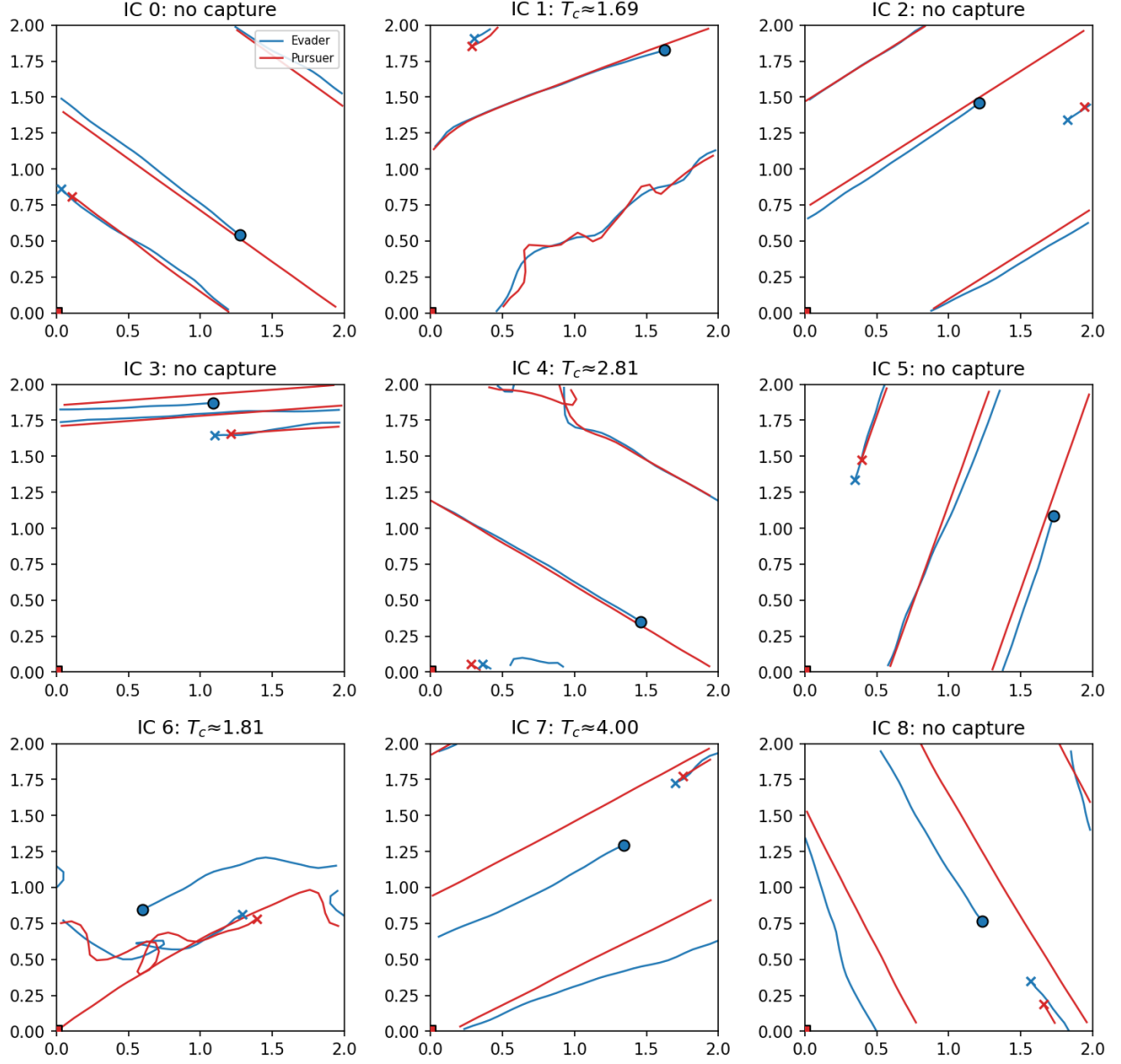


Figure 3: Learned trajectories of the evader (blue) and the pursuer (red) for nine of the twelve initial conditions used in training. Squares mark the starting positions; crosses mark the final positions at $t = t_{\max}$. Line breaks indicate wrap-around at the torus boundary. Captures (when they occur) take place at the smallest k for which $\|\Delta_{\mathbb{T}}^2\|_k \leq \varepsilon$; T_c is reported in the panel title.

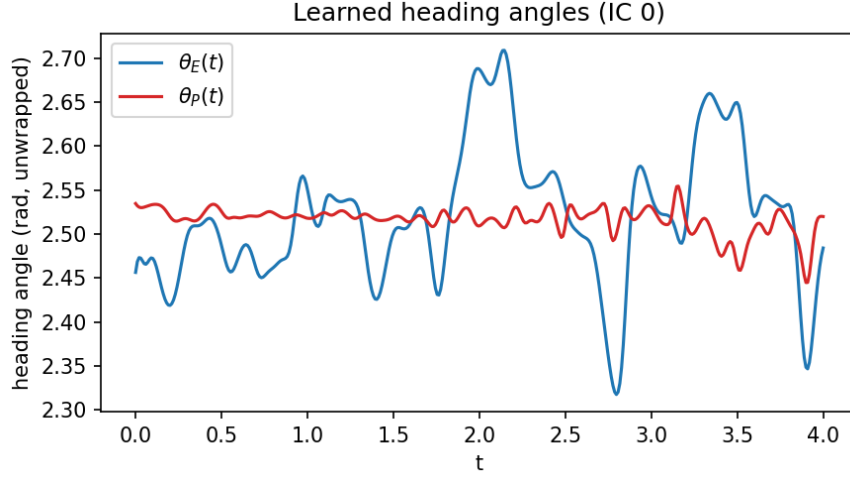


Figure 4: Unwrapped heading angles $\theta_E(t)$ and $\theta_P(t)$ for one representative initial condition. Smooth monotone-like profiles indicate that the open-loop policies converge to physically sensible trajectories.

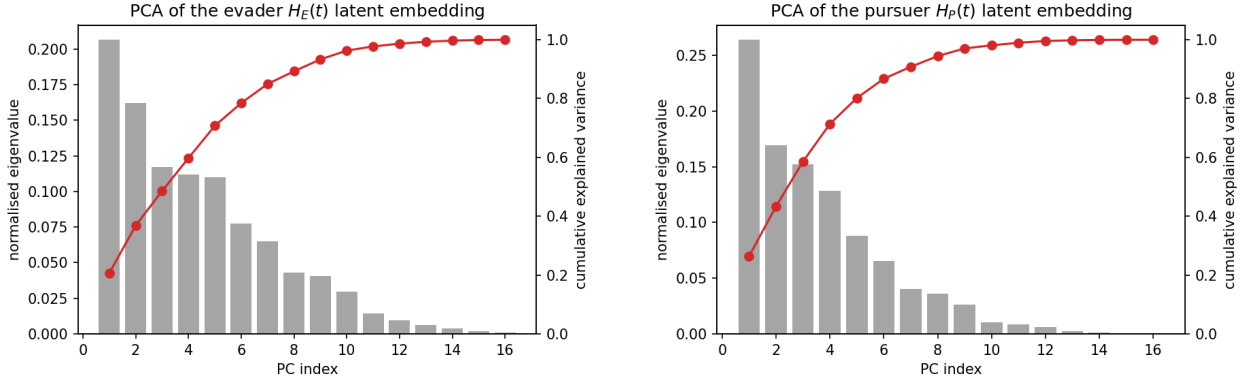


Figure 5: PCA of the centred latent embeddings $H_E(t)$ (left) and $H_P(t)$ (right) over a dense time grid. Bars: normalised eigenvalues; solid line: cumulative explained variance. A handful of components capture the bulk of the variance, indicating an effectively low-dimensional manifold of optimal time-parametrised strategies.